Pascal Subset Mini-Compiler

Project Report

Session 2023 - 2027

Submitted by:

Hamna Arshad 2023-CS-63

Saroosh Javed 2023-CS-67

Submitted to:

Ma’am Aliya Farooq

Course:

CSC-303L Compiler Construction B

Department of Computer Science

University of Engineering and Technology

Lahore Pakistan

Table of Contents

List of Tables

List of Figures

1. Introduction

A compiler translates source code into a form the machine can use. This project covers the front end of that process, which is everything that happens before code generation. The compiler answers three basic questions about a program:

- Are the tokens spelled correctly? (lexical analysis)
- Does the syntax follow the grammar rules? (parsing)
- Is the program meaningful? (semantic analysis)

The target language is a Pascal subset. It is small enough to finish in one semester, but it still includes variables, functions, arrays, control flow, and expressions.

2. Project Objectives

The project was designed to meet the following goals:

- Implement a lexer with accurate line and column tracking.
- Compute FIRST, FOLLOW, and the LL(1) table from a grammar file.
- Build three parsers and compare them on the same input.
- Construct an AST and symbol table through recursive descent.
- Apply semantic rules for types, assignments, and function calls.
- Deliver both a command-line tool and a graphical interface.

3. Language Scope

3.1 Supported Features

Table 1: Information of Supported Features

Feature	Details
Program structure	program header, var block, begin ... end
Types	integer, real, array [lo..hi] of type
Subprograms	function and procedure with parameters
Statements	assignment, if/then/else, while/do
Expressions	arithmetic, relations, not, parentheses

4. System Overview

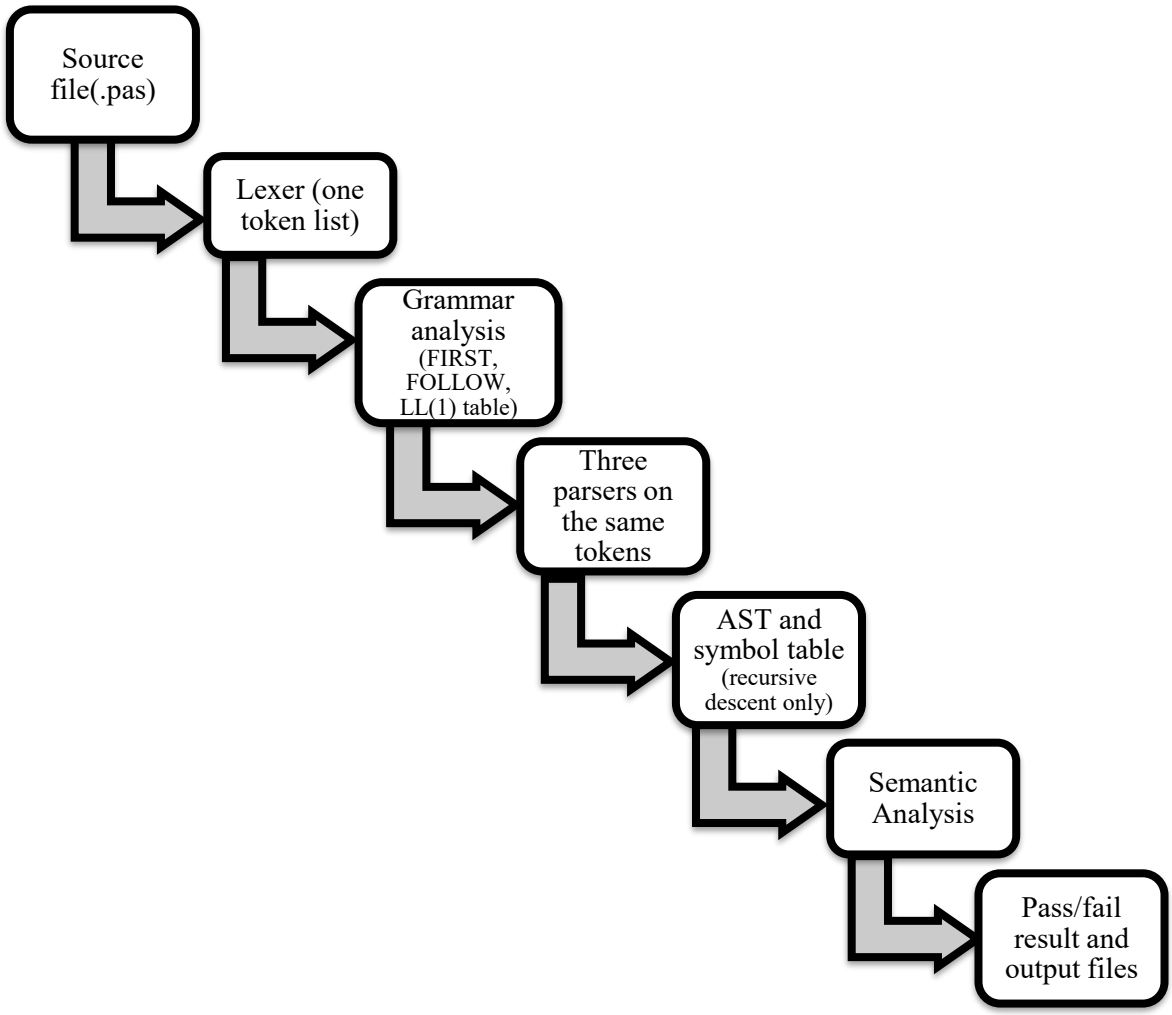

Figure 1: Overall Compilation Pipeline

Table 2: User interfaces built on the same compiler core

Component	Role
mini_compiler (CLI)	Runs phases individually or as a full pipeline
compiler_gui (Qt 6)	Visual display of tokens, tables, traces, and AST

5. Project Structure

Table 3: Main project folders.

Folder	Responsibility
src/lexer/	Input buffer and token scanner
src/grammar_analysis/	FIRST, FOLLOW, LL(1) table
src/parser/	Recursive descent, LL(1), and SLR parsers
src/ast/	Syntax tree nodes
src/symbol_table/	Scoped symbol storage
src/semantic/	Type and usage checking
gui/	Qt visualization application
test/	Sample and validation programs

6. Lexical Analysis

Lexical analysis converts the character stream into tokens. The design has three layers:

6.1 Double Buffer

The input stage uses a double buffer as taught in the lab:

- The file is read in 4096-character chunks.
- Two buffers alternate so disk access stays efficient.
- A sentinel marks the end of each buffer half.

6.2 Scanner

Table 4: Main scanner rules.

Input pattern	Token produced
Letter-led words	Keyword or identifier
Digit sequences	Integer or real literal
:=	Assignment operator
..	Range operator
Relational symbols	Relational operator
{ ... }	Comment (skipped)

Each token records its kind, lexeme, attribute, line number, and column number for error reporting.

6.3 Token Stream

Parsers access tokens through a TokenStream wrapper:

- peek() and advance() control reading order.
- Token kinds map to grammar terminal names.
- The dot after end is normalized before end-of-file.

The lexer runs once. Each parser receives a fresh TokenStream from the same list. Parsing stops if lexical errors are found.

7. Grammar Analysis

Table 5: Grammar files for the same language

Grammar file	Used by	Style
grammar_ll1.txt	RD, LL(1), FIRST/FOLLOW, LL(1) table	Right-recursive
grammar_original.txt	SLR parser	Left-recursive

Both files describe the same Pascal subset. The rule shape changes to fit top-down or bottom-up parsing.

7.1 FIRST and FOLLOW

The compiler computes these sets automatically:

- FIRST: terminals that can begin a non-terminal.
- FOLLOW: terminals that can appear after a non-terminal.

7.2 LL(1) Table

The predictive table is built from those sets:

- Each cell maps (non-terminal, lookahead) to one production.
- Conflicts are reported when two productions compete.

8. Parsing

Three parsing techniques are implemented so their behavior can be compared on identical input. Workflow figure shows that they run in parallel on the same token list, not one after another.

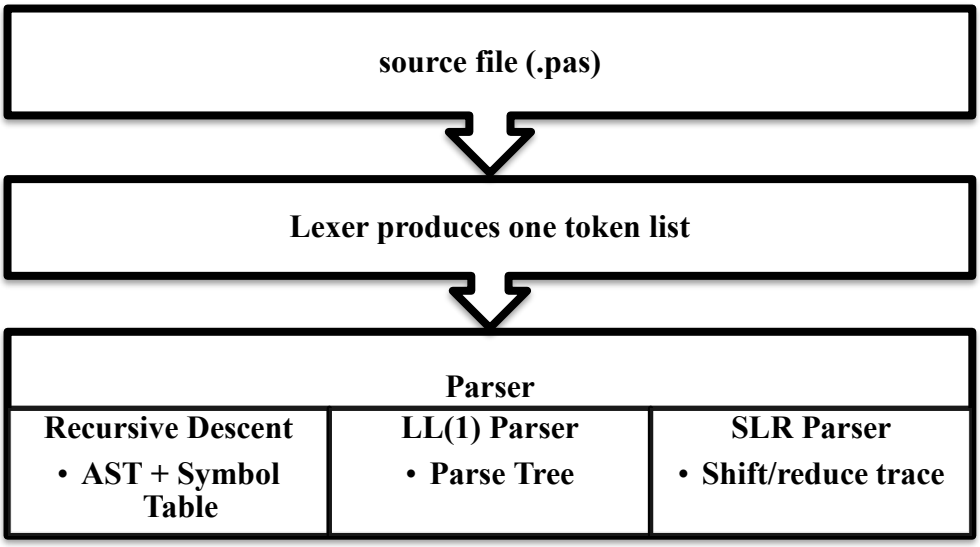

Figure 2: Three parsers share the same source and tokens

Table 6: Parser comparison.

Parser	Direction	Builds AST	Grammar file
Recursive descent	Top-down	Yes	grammar_ll1.txt
LL(1)	Top-down (table)	No	grammar_ll1.txt
SLR	Bottom-up	No	grammar_original.txt

8.1 Recursive Descent

This parser is the main path for AST and symbol table construction:

- One function per grammar non-terminal.
- Code structure follows the grammar closely.
- Declarations populate the symbol table during parsing.

8.2 LL(1) Parser

This parser demonstrates table-driven prediction:

- Uses a stack and the precomputed LL(1) table.
- Records each step in a parse trace for the GUI.

8.3 SLR Parser

This parser shows bottom-up parsing:

- Uses ACTION and GOTO tables from SLR item sets.
- Reads the left-recursive grammar file.
- Produces a shift/reduce trace.

9. Abstract Syntax Tree

The AST keeps meaningful program structure and leaves out helper grammar rules. Main node types include:

- Program, VarDecl, Assign, If, While, FuncCall, and related nodes.
- Expression nodes carry inferred types after semantic analysis.
- The GUI provides both a tree view and a zoomable graph view.

10. Symbol Table

Table 7: Fields stored per symbol

Field	Content
Name	Identifier text
Kind	Variable, function, procedure, or parameter
Type	integer, real, or array type
Scope	Nesting level
Line	Declaration location
Params	Argument count and types for subprograms

Scopes open on begin blocks and subprogram bodies. Lookup checks inner scopes before outer ones.

11. Semantic Analysis

Parsing checks syntax. Semantic analysis checks meaning by walking the AST:

Table 8: Semantic rules enforced

Check	Rule
Arithmetic	integer + integer gives integer; real promotes result

Assignment	integer := real is rejected
Function call	Argument count and types must match
Array access	Index must be integer; target must be an array
Procedure use	A procedure name cannot appear as a value
Undeclared name	Reported as a semantic error

11.1 Test Files

Table 9: Semantic validation programs.

File	Purpose
sem_ok_mixed.pas	Valid mixed-type program
sem_err_int_from_real.pas	Invalid assignment
sem_err_arity.pas	Wrong argument count
sem_err_array_index.pas	Invalid index type
sem_err_proc_in_expr.pas	Procedure used in expression

12. Error Handling

Table 10: Error categories

Category	Example
Lexical	Invalid character in source
Syntactic	Missing semicolon or unexpected token
Semantic	Type mismatch or undeclared identifier

The error system provides the following behavior:

- Each error includes line, column, and a short message.
- Panic-mode recovery can report multiple syntax errors in one run.

The GUI jumps to the error line when a row is double-clicked.

13. Command-Line Interface

The mini_compiler tool supports these common commands:

- ./mini_compiler file.pas --lexer
- ./mini_compiler --first
- ./mini_compiler --ll1-table
- ./mini_compiler file.pas --rd
- ./mini_compiler file.pas --ll1
- ./mini_compiler file.pas --lr
- ./mini_compiler file.pas --full --out output/full_compile.txt

A full compile also writes separate files to output/: tokens.txt, first_follow.txt, ll1_table.txt, ll1_trace.txt, lr_table.txt, and lr_trace.txt.

14. Graphical User Interface

The Qt 6 GUI displays results from CompilerSession. It does not duplicate compiler logic.

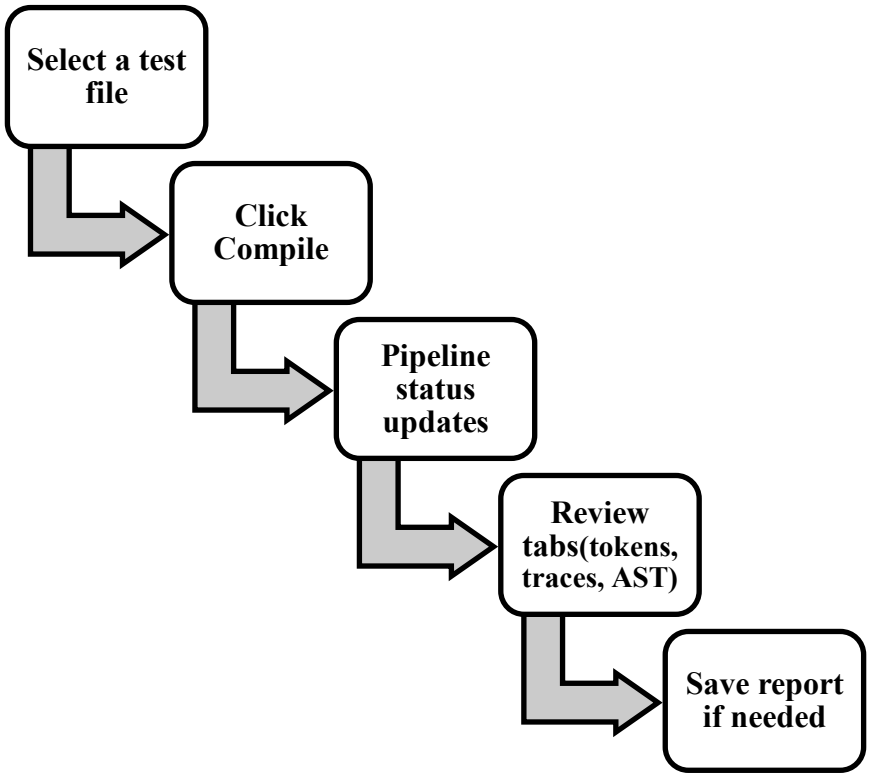

Figure 3: Typical demonstration flow in the GUI

Table 11: GUI layout

Panel	Content
Test Explorer	List of .pas test files
Source and AST	Editor plus tree and graph views
Compiler Results	Pipeline status, result tabs, and metrics

The results area includes tabs for:

- Tokens, FIRST/FOLLOW, and the LL(1) table.
- Recursive descent, LL(1), and LR parse traces.
- Symbol table, semantic errors, and a full report export.

15. Testing

The main test files and their expected outcomes are listed below:

Table 12: Primary test cases

Test file	Expected result
sample1.pas	All parsers accept; no semantic errors
valid_full.pas	Functions, if, and while compile correctly
sample2.pas	Parses successfully but fails semantics
err_lex.pas	Lexical error detected
err_syn.pas	Syntax error detected

The build was verified on WSL Ubuntu with GCC C++17 and Qt 6.

16. Results and Observations

Implementation and testing led to the following observations:

- Lexer accuracy affects every later stage. One bad token rule can break parsing.
- The LL and LR grammars look different but accept the same programs.
- Recursive descent is the most direct way to build an AST.
- LL(1) and SLR traces are useful for explaining parsing steps in class.
- Semantic checks catch errors that syntax checking alone cannot find.

17. Limitations

The current version has these known limits:

- No intermediate code or machine code generation.
- Coverage is limited to the defined Pascal subset.
- Array bounds are not verified against declared ranges.
- The GUI requires a separate Qt 6 installation.

18. Conclusion

This project delivers a complete compiler front end: lexer, grammar analysis, three parsers, AST, symbol table, and semantic validation. The same pipeline runs in both the CLI and the GUI. The main result is a single integrated system where course topics work together and can be inspected at each stage.